รายงาน

Agile Modeling, Agile Process, Scrum

เสนอ

ผศ. ดร. ธนันชัย คำเกตุ

จัดทำโดย

นาย เสฏฐวุฒิ แก้วก่า 66011211019

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา Software Engineering 1201323

ภาคเรียนที่ 2 ปีการศึกษา 2568

คณะวิทยาการสารสนเทศ สาขาเทคโนโลยีสารสนเทศ มหาวิทยาลัยมหาสารคาม

ประวัติความเป็นมา

ระหว่างวันที่ 11–13 กุมภาพันธ์ ค.ศ. 2001 คน 17 คนนัดพบกันที่รีสอร์ทที่ชื่อว่า Snowbird Ski ที่อยู่ท่ามกลางภูเขา Wasatch ในรัฐยูท่าห์ ประเทศสหรัฐอเมริกา เพื่อพูดคุย เล่นสกี พักผ่อน ทานอาหาร และหาจุดร่วมรูปแบบการทำงานแบบใหม่ สิ่งที่ได้จากการพบกันครั้งนี้คือคำประกาศของการพัฒนาซอฟต์แวร์แบบ Agile นั่นเอง มีตัวแทนจากรูปแบบในการทำงานแบบ Extreme Programming, SCRUM, DSDM, Adaptive Software Development, Crystal, Feature-Driven Development, Pragmatic Programming และอื่นๆ ต่างร่วมถกกันเพื่อให้เกิดทางเลือกใหม่ที่เข้ามาแทนที่การพัฒนาซอฟต์แวร์แบบเก่าที่เรียกว่า **Heavyweight**

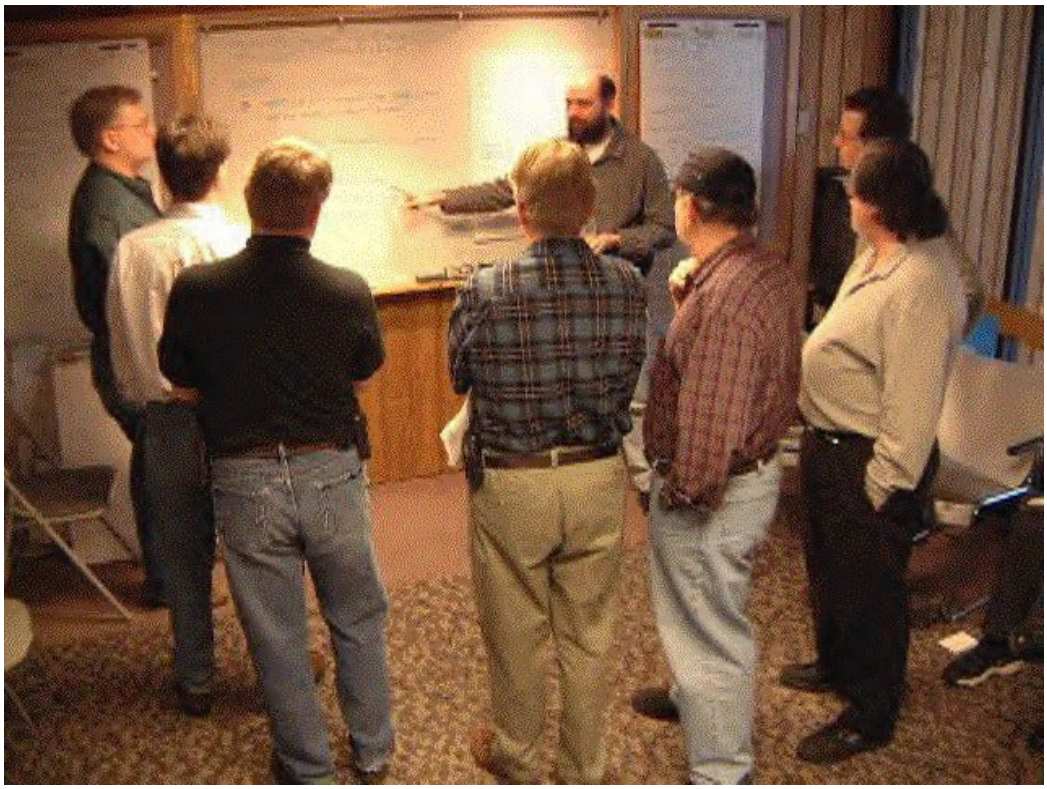
การรวมตัวกันของคนจำนวนเยอะ ๆ เช่นนี้ที่มีความเห็นต่างออกไปของความเป็นไปในองค์กรในโลกปัจจุบันนั้นค่อนข้างที่จะหายาก ดังนั้นการรวมตัวกันในครั้งนี้ทำให้เกิด — คำประกาศการพัฒนาซอฟต์แวร์แบบ **(Agile (Manifesto for Agile Software Development)** — โดยมีการลงนามโดยผู้เข้าร่วมประชุมทั้งหมด โดย Martin Fowler นักพัฒนาซอฟต์แวร์ชาวอังกฤษมีความเป็นห่วงว่าคนอเมริกันส่วนใหญ่จะออกเสียงคำว่า Agile ไม่ถูกต้อง

ความกังวลช่วงต้นๆ ของการประชุมของ **Alistair Cockburn** สะท้อนถึงความคิดของผู้เข้าร่วมหลายๆ คนที่คิดว่า ตัวเขาเองไม่ได้คาดหวังว่าผู้เข้าร่วมประชุมจะมีการตกลงเรื่องอะไรเป็นชิ้นสำคัญมากนัก แต่หลังจากที่ประชุมเสร็จ Alistair ได้ให้ความเห็นว่า เขารู้สึกดีใจมากกับคำประกาศการพัฒนาซอฟต์แวร์แบบ **Agile** เขามีความประหลาดใจที่ทุกคนก็ดีใจกับคำประกาศนี้ ดังนั้นเราได้มีการตกลงเรื่องสำคัญในการประชุมในครั้งนี้

เราได้ตั้งชื่อกลุ่มขึ้นมา นั่นคือ “**The Agile Alliance**” เป็นกลุ่มของคนที่มีความคิดเป็นอิสระในการพัฒนาซอฟต์แวร์ ในขณะเดียวกันคนกลุ่มนี้ก็กลายเป็นคู่แข่งกันเอง คนกลุ่มนี้ได้ตกลงกันใน คำประกาศการพัฒนาซอฟต์แวร์แบบ Agile ที่อยู่ในเว็บไซต์นี้

ในขณะที่คำประกาศ (Manifesto) ได้ให้ความคิดหรือไอเดียที่เฉพาะเจาะจง ก็มีไอเดียที่ลึกกว่าคำประกาศที่มีผลต่อผู้เข้าประชุมหลายๆ คน (ไม่ใช่ทุกคน) เช่นกัน Bob Martin ได้พูดเชิงตลกไว้ตอนท้ายหลังจากประชุมสองวันว่า เขากำลังเขียนคำประกาศในเชิงของอารมณ์และความรู้สึก (Mushy Statement) มีเพียงบางคนเท่านั้นที่ไม่เห็นด้วยกับเขา พวกเราทุกคนรู้สึกว่าได้รับสิทธิพิเศษที่ได้ร่วมงานกับกลุ่มคนที่มีความคิดเห็นตรงกัน ได้ร่วมกันเขียนคุณค่า (Value) ที่ว่าด้วยเรื่องความไว้ใจและการให้ความเคารพซึ่งกันและกัน ได้สนับสนุนรูปแบบองค์กรที่ให้ความสำคัญกับคน (people) ความร่วมมือ (collaboration) และการสร้างองค์กรที่เราต้องการไป

ร่วมงานด้วย โดยผม (Jim Highsmith) ก็มีความเชื่อว่า Agile ประกอบไปด้วยความรู้สึกนึกคิด (Mushy Stuff) ที่หมายถึงการส่งมอบสินค้าที่ดีให้กับลูกค้า โดยการจัดสภาวะในการทำงาน โดยไม่มานั่งพูดกันว่าคนทำงานเป็นทรัพย์สมบัติของบริษัทที่สำคัญที่สุด แต่ที่จริงแล้วคนเป็นสิ่งที่สำคัญที่สุด โดยตัดคำว่าทรัพย์สมบัติออก จากความสนใจที่มีต่อ Agile ที่มากขึ้นและข้อวิพากษ์วิจารณ์ต่อ Agile ก็มีมากเช่นกัน สุดท้ายคือแนวคิด Agile คือสิ่งที่ประกอบไปด้วยความรู้สึกนึกคิด (Mushy Stuff) ที่ว่าด้วยเรื่องของคุณค่า (Values) และวัฒนธรรม (Cultures)



ตัวอย่างเช่น ผม (Jim Highsmith) คิดว่า Extreme Programming ได้รับความนิยมอย่างรวดเร็วไม่ได้เกิดจากการที่มี Pair-Programming (การเขียนโปรแกรมโดยใช้โปรแกรมเมอร์สองคน) หรือแม้แต่ Refactoring (การเปลี่ยนโครงสร้างภายในของรหัสคอมพิวเตอร์) แต่ที่จริงแล้วต้องดูจากภาพรวมทั้งหมดที่ชุมชนของนักพัฒนาโปรแกรม **(Developer Community)** เป็นอิสระจากองค์กรที่เรียกว่า **Dilbertesque corporation** (องค์กรที่มีการโปรโมทให้พนักงานที่ไม่มีประสิทธิภาพในการทำงานในหน้างานเชิงเทคนิคของตนเองขึ้นไปเป็นระดับหัวหน้างานหรือระดับบริหารเพื่อป้องกันว่าผู้นั้นจะไม่ทำผิดพลาดในงานเชิงเทคนิคของตนเอง งานของเขาเหล่านั้นก็คือออกคำสั่งให้คนที่ทำงานเก่งให้ทำงานต่าง ๆ แต่ผู้นั้นก็ไม่มีทักษะของการเป็นผู้นำในการบริหารงานและบริหารคนเลย) Ken Beck เล่าให้ฟังว่า ตอนที่เขาเริ่มต้นทำงานใหม่ๆ นั้น เขาได้ประเมินว่าการเขียนโปรแกรมในโปรเจ็คนั้น

คาดว่าใช้เวลา 6 สัปดาห์ และใช้คนทำ 2 คน หลังจากนั้นผู้จัดการของเขาได้ให้นักเขียนโปรแกรมอีกคนหนึ่งไปทำโปรเจ็คอื่น ทำให้เขาต้องใช้เวลากว่า 12 สัปดาห์ถึงจะทำโปรเจ็คนั้นให้แล้วเสร็จ โดยที่เขารู้สึกแย่กับตัวเองมาก ยิ่งไปกว่านั้น ผู้จัดการของเขากร่นด่าว่าเขาทำงานช้ามากหลังจากที่ผ่านช่วง 6 สัปดาห์แรกไปแล้ว เขารู้สึกไม่ดีเอามากๆ เพราะเขาคิดว่านั้นเป็นความผิดพลาดของเขาเอง แต่สุดท้ายแล้วเขาพบว่าโปรเจ็คนี้ต้องใช้คน 2 คน ถึงจะทำเสร็จภายในระยะเวลา 6 สัปดาห์ ดังนั้นการทำงานไม่ได้ตามเป้าหมายครั้งนี้ของเขาคือความผิดพลาดของผู้จัดการของเขานั่นเอง นับว่าเป็นความล้มเหลวที่เกิดจากความคิดที่ไม่ยอมปรับเปลี่ยนที่มักเกิดขึ้นบ่อยๆ กับวงการไอที

สถานการณ์แบบนี้เกิดขึ้นได้ทุกวัน ไม่ว่าจะเป็นคนในวงการการตลาด ผู้บริหาร ลูกค้าภายนอก ลูกค้าภายใน และแน่นอนนักพัฒนา ที่ผู้คนเหล่านี้ไม่อยากตัดสินใจกับอะไรที่มันยาก ๆ เลยสั่งการอย่างไร้เหตุผลผ่านโครงสร้างของบริษัทแทน นี่ไม่ได้เป็นปัญหาเฉพาะของวงการพัฒนาซอฟต์แวร์เท่านั้น แต่เป็นปัญหาของทุกองค์กรที่เป็นแบบ **Dilbertesque Organization**

การที่จะประสบความสำเร็จในเศรษฐกิจยุคใหม่ที่ก้าวเข้าสู่ e-business, e-commerce, และเว็บไซต์ บริษัทต่างๆ ต้องเอาตัวเองออกจากการบริหารงานแบบ Dilbert ซึ่งการมีชีวิตในการทำงานที่มีอิสระมากยิ่งขึ้นเป็นสิ่งดึงดูดให้เกิดแนวคิดของ Agile และสิ่งนี้ก็ทำให้คนที่นิยมการทำงานแบบเดิมๆ เริ่มกลัวกับการเปลี่ยนแปลงที่จะเกิดขึ้น เป็นที่แน่นอนว่าการบริหารงานแบบ Agile ทำให้องค์กรที่ทำงานแบบเดิมๆ สั่นคลอนและหายไป องค์กรกรเหล่านี้จะมีการบริหารงานแบบกดดันมากกว่าที่จะพยายามทำให้ดีที่สุดให้กับลูกค้าพึงพอใจและสิ่งมอบตรงเวลาตามที่ได้ตกลงกันไว้

ความเคลื่อนไหวของ Agile ไม่ได้ถูกสร้างขึ้นมาเพื่อต่อต้านกับระบบการทำงานแบบระบบเดิมแต่เป็นการกู้เครดิตให้กับระบบเดิมซะมากกว่า พวกเราต้องการที่จะทำให้เกิดความสมดุลขึ้น เรารับได้กับการทำแบบจำลอง แต่ไม่ใช่แบบจำลองที่ไม่ได้ประโยชน์อะไรจนฝุ่นจับ เรายอมรับกับการทำเอกสารแต่ไม่ใช่เอกสารที่ต้องทำหลายร้อยหน้าที่ไม่เคยได้รับการใช้หรือดูแล เรามีการวางแผนงาน แต่เป็นการวางแผนงานที่ไม่สมบูรณ์แบบแต่ดีพอสำหรับธุรกิจที่อยู่ในสภาวะที่ผันผวน มีหลาย ๆ คนที่นำเอา XP (Extreme Programming) หรือ SCRUM หรือ รูปแบบการทำงานอื่น ๆ ไปใช้แต่ไม่ได้เข้าใจรูปแบบการทำงานแบบนั้นอย่างลึกซึ้ง

ก่อนหน้าที่จะมีการประชุมที่ Snowbird Ski ที่รัฐยูท่านั้น ที่จริงแล้วมีการประชุมของกลุ่มคนที่ทำงานภายใต้รูปแบบ Extreme Programming และผู้ที่สนใจ จัดขึ้นโดย Ken Beck ที่ Rogue River Lodge ที่รัฐโอเรกอน ในช่วงฤดูใบไม้ผลิ ปี ค.ศ. 2000 ในที่ประชุมผู้เข้าร่วมมีการพูดถึงเพื่อสนับสนุนการทำงานภายใต้รูปแบบที่เรียกว่า Light Methodologies แต่เมื่อจบการประชุมก็ไม่ได้มีอะไรผลลัพธ์อะไรออกมาเป็นทางการ ในช่วงปี ค.ศ. 2000 นั้นมีบทความมากมายที่เขียนถึงการทำงานในรูปแบบที่เรียกว่า Light หรือLightweight ซึ่งนั่นก็คือการทำงานในรูปแบบของ Extreme Programming, Adaptive Software Development, Crystal, และ SCRUM ในช่วงของการสนทนาที่รัฐโอเรกอนนั้นก็ไม่มีใครอยากใช้คำว่า Light แต่ก็ยังพูดเป็นที่ติดปากในเวลานั้น

เดือนกันยายนปี 2000 Bob Martin จาก Object Mentor ในชิคาโก้ ได้ส่งอีเมล์ โดยมีเนื้อหาดังนี้ “ผมอยากจะจัดประชุมเล็ก ๆ สักสองวันในช่วงเดือนมกราคมถึงเดือนกุมภาพันธ์ ค.ศ. 2001 ที่ชิคาโก้ วัตถุประสงค์ของการประชุมคืออยากให้เจ้าของการทำงานแบบใหม่ หรือ Lightweight ได้มาประชุมชุมนุมในห้องเดียวกัน ทุกคนได้รับการรับเชิญให้เข้าประชุมในครั้งนี้ ผมอยากทราบว่าผมควรจะเชิญใครอีก” แล้ว Bob Martin ก็ไปทำ Wiki ขึ้นมาที่เกิดการอภิปรายกันในนั้นอย่างมากมาย

ตอนแรก Alistair Cockburn ตอบกลับมาว่าเขาไม่ชอบคำว่า **Light** มากนัก เขาพูดว่า “เขาไม่ติดใจที่จะเรียกการทำงานในรูปแบบที่ **Light** แต่ผมไม่แน่ใจว่าจะให้เรียกผมว่าเป็นผู้ที่มีน้ำหนักน้อย ผอมแห้ง **(Lightweight)** เข้าร่วมประชุมกันว่าด้วยเรื่องผอมแห้ง และก็เหมือนการประชุมที่ผู้เข้าร่วมประชุมหลาย ๆ คนพยายามจำวันว่าวันนั้นเป็นวันอะไร”

เมื่อการถกเถียงในเรื่องของสถานที่จัดประชุมได้สิ้นสุดลง ก็มีความเป็นกังวลของสภาพอากาศในฤดูหน้าที่เมืองชิคาโกว่าอาจจะหนาวมากจนไม่สามารถทำอะไรที่สนุก ๆ ได้ จึงได้มีการจัดการประชุมขึ้นที่รัฐยูทาห์ขึ้นแทน ถึงแม้ว่าที่นั่นจะหนาวแต่ก็มีอะไรสนุก ๆ ให้ทำ เช่นการเล่นสกี ซึ่ง Martin Fowler ก็เล่นตั้งแต่ไปถึงวันแรกเลย อีกที่หนึ่งคือ Anguilla ในทะเลแคริบเบียน ที่นั่นอากาศอุ่นและมีสิ่งให้ทำสนุกสนาน แต่ก็จะเสียเวลาในการเดินทางไปที่นั่น สุดท้ายแล้วสถานที่เล่นสกีที่ชื่อว่า Snowbird จึงถูกเลือก อย่างไรก็ตามบางคนอย่างเช่น Ron Jeffries อยากให้ไปจัดประชุมที่ที่มีอากาศอบอุ่นในครั้งหน้า

พวกเราหวังว่าการทำงานของพวกเราในครั้งนี้ในฐานะ **Agile Alliance** จะช่วยให้คนอื่น ๆ ได้คิดคำนึงถึงการทำงานพัฒนาซอฟต์แวร์ และรูปแบบในการทำงานแบบต่าง ๆ รวมไปถึงรูปแบบองค์กร ในรูปแบบใหม่ที่เรียกว่า รูปแบบ Agile ถ้าสิ่งนี้เกิดขึ้นได้จริงพวกเราก็บรรลุวัตถุประสงค์ของพวกเราแล้ว

หลักการของ Agile

ความหมายของ Agile

Agile คือแนวคิดและแนวทางการพัฒนาระบบ/ซอฟต์แวร์ที่มุ่งเน้นการทำงานแบบยืดหยุ่น (Flexible) ปรับตัวได้รวดเร็วต่อการเปลี่ยนแปลง และส่งมอบผลงานเป็นรอบสั้น ๆ (Iterative/Incremental) เพื่อให้ผู้ใช้หรือผู้มีส่วนได้ส่วนเสียได้เห็นผลลัพธ์จริงอย่างต่อเนื่อง พร้อมรับข้อเสนอแนะ (Feedback) เพื่อนำไปปรับปรุงให้ตรงความต้องการมากที่สุด

ค่านิยมหลัก (Agile Values)

- Agile มีค่านิยมหลัก 4 ข้อที่ใช้เป็นแนวทางการตัดสินใจในการทำงาน
- ให้ความสำคัญกับคนและการสื่อสาร มากกว่า กระบวนการและเครื่องมือ
- ให้ความสำคัญกับซอฟต์แวร์ที่ใช้งานได้จริง มากกว่า เอกสารที่ครบถ้วนจำนวนมาก
- ให้ความสำคัญกับความร่วมมือกับลูกค้า/ผู้ใช้ มากกว่า การต่อรองตามสัญญา
- ให้ความสำคัญกับการตอบสนองต่อการเปลี่ยนแปลง มากกว่า การทำตามแผนตายตัว

หลักการสำคัญ (Agile Principles) แบบสรุปใช้งานรายงาน

- หลักการของ Agile สามารถสรุปเป็นประเด็นสำคัญที่พบบ่อยในการใช้งานจริง ดังนี้
- ส่งมอบคุณค่าเร็วและสม่ำเสมอ โดยแบ่งงานเป็นชิ้นเล็ก ๆ และส่งมอบเป็นรอบ
- ยอมรับการเปลี่ยนแปลงของความต้องการ แม้จะเกิดขึ้นช่วงท้ายของโครงการ
- ทำงานร่วมกันอย่างใกล้ชิดระหว่างทีมพัฒนาและผู้เกี่ยวข้อง เพื่อให้ความต้องการชัดเจน
- สื่อสารบ่อยและโปร่งใส ลดความเข้าใจผิด และช่วยแก้ปัญหาได้เร็ว
- ตรวจสอบและปรับปรุงอย่างต่อเนื่อง (Continuous Improvement) ผ่านการทบทวนผลการทำงาน
- เน้นคุณภาพของงาน ทั้งด้านการออกแบบ การทดสอบ และความพร้อมใช้งานของระบบ
- ทำงานแบบยั่งยืน ไม่เร่งจนเกินไปจนเกิดความผิดพลาดสะสม (ลด Technical Debt)

องค์ประกอบของ Agile

1) การทำงานเป็นรอบ (Iterations / Increments)

Agile มักแบ่งการทำงานเป็นรอบเวลาสั้น ๆ เช่น 1–4 สัปดาห์ เพื่อพัฒนาและส่งมอบ “ชิ้นงานที่ใช้งานได้” ในแต่ละรอบ ทำให้สามารถตรวจสอบคุณภาพและรับข้อเสนอแนะได้ต่อเนื่อง และช่วยลดความเสี่ยงจากการรอส่งงานครั้งเดียวเมื่อจบโครงการ

2) การส่งมอบและรับข้อเสนอแนะ (Frequent Delivery & Feedback)

หัวใจของ Agile คือการส่งมอบผลงานที่ใช้งานได้จริงอย่างสม่ำเสมอ จากนั้นนำ Feedback จากผู้ใช้/ผู้เกี่ยวข้องมาปรับปรุงระบบทันที ทำให้ระบบมีแนวโน้มตรงกับความต้องการจริงมากขึ้น และลดงานที่ทำแล้วไม่ตอบโจทย์

3) การจัดลำดับความสำคัญของงาน (Prioritization)

Agile ใช้แนวคิดการเรียงลำดับงานตาม “คุณค่าทางธุรกิจ” และ “ความสำคัญต่อผู้ใช้” เช่น การทำรายการงาน (Backlog) และเลือกทำสิ่งที่ให้ประโยชน์สูงก่อน เพื่อให้โครงการคุ้มค่าและเห็นผลเร็ว

4) ทีมทำงานร่วมกัน (Collaboration & Cross-functional Team)

ทีม Agile มักเป็นทีมขนาดเล็กถึงกลางและทำงานร่วมกันแบบข้ามสายงาน (เช่น วิเคราะห์ ออกแบบ พัฒนา ทดสอบ) เน้นการช่วยเหลือกันและรับผิดชอบร่วมกันต่อผลลัพธ์ ทำให้การแก้ปัญหาและการตัดสินใจรวดเร็วขึ้น

5) ความโปร่งใสและการสื่อสาร (Transparency & Communication)

Agile ให้ความสำคัญกับการสื่อสารที่ชัดเจนและเห็นภาพรวมงาน เช่น การประชุมสั้นรายวัน การรายงานความคืบหน้า การทำบอร์ดงาน (Kanban/Scrum board) เพื่อให้ทีมและผู้เกี่ยวข้องรับรู้สถานะงาน ลดการทำงานซ้ำซ้อน และตรวจพบปัญหาได้เร็ว

6) การปรับปรุงอย่างต่อเนื่อง (Continuous Improvement)

เมื่อจบรอบการทำงาน ทีมจะทบทวนสิ่งที่ทำได้ดีและสิ่งที่ต้องปรับปรุง (เช่น Retrospective) แล้วนำไปพัฒนากระบวนการทำงานรอบถัดไป ส่งผลให้ทีมทำงานมีประสิทธิภาพขึ้นเรื่อย ๆ

7) คุณภาพของซอฟต์แวร์ (Quality Practices)

Agile สนับสนุนให้รักษาคุณภาพของระบบอย่างต่อเนื่อง เช่น การทดสอบสม่ำเสมอ การตรวจโค้ด (Code Review) การรวมโค้ดเป็นระยะ และการปรับโครงสร้างโค้ดเมื่อจำเป็น (Refactor) เพื่อให้ระบบขยายต่อได้ในอนาคต

เทคนิคการพัฒนา (Agile Development Techniques)

1) การเก็บความต้องการแบบ User Story

Agile มักแปลงความต้องการเป็น User Story เพื่อให้ทีมเข้าใจจากมุมมองผู้ใช้ เช่น
As a (ผู้ใช้)... I want (ต้องการ)... so that (เพื่อให้)...
โดยกำหนด Acceptance Criteria (เงื่อนไขการยอมรับ) ให้ชัดเจนเพื่อใช้เป็นเกณฑ์ทดสอบและยืนยันงาน

2) การจัดการงานด้วย Backlog และการจัดลำดับความสำคัญ

ใช้รายการงาน เช่น Product Backlog / Sprint Backlog เพื่อรวบรวมงานทั้งหมด และจัดลำดับตามคุณค่า/ความเร่งด่วน เช่น

- ทำฟีเจอร์หลักที่สร้างคุณค่าก่อน
- ลดความเสี่ยงด้วยการทำส่วนที่ไม่แน่นอนก่อน
- แยกงานเป็นชิ้นเล็ก (Small batch) เพื่อเสร็จได้เร็วและติดตามง่าย

3) การประมาณการงาน (Estimation)

การประเมินความยากของงานเพื่อวางแผนรอบการทำงาน เช่น

- Story Points (คะแนนความยาก)
- Planning Poker (โหวตร่วมกันเพื่อให้ได้ค่ากลาง)
- ช่วยให้ทีมวางแผนปริมาณงานในแต่ละรอบได้เหมาะสม

4) การพัฒนาแบบ Incremental & Iterative

พัฒนาระบบ “เพิ่มทีละส่วน” และ “ปรับปรุงซ้ำเป็นรอบ” โดยทำให้แต่ละรอบได้ชิ้นงานที่ใช้งานได้จริง (Increment) ลดความเสี่ยงจากการทำครั้งเดียวแล้วค่อยรวมตอนท้าย

5) การทดสอบอย่างต่อเนื่อง (Testing Practices)

แนวทางที่นิยมใน Agile เพื่อรักษาคุณภาพงาน

- Unit Test / Integration Test
- Test-Driven Development (TDD) (บางทีมใช้)
- Regression Test เพื่อป้องกันบัคกลับมา หลักคือ “ทดสอบบ่อย” และทำให้ปัญหาเจอเร็ว แก้เร็ว

6) การปรับปรุงคุณภาพโค้ด (Refactoring)

การปรับโครงสร้างโค้ดให้ดีขึ้นโดยไม่เปลี่ยนพฤติกรรมของระบบ เพื่อให้อ่านง่าย ดูแลง่าย และขยายระบบในอนาคตได้ ลดปัญหา Technical Debt

7) Continuous Integration / Continuous Delivery (CI/CD)

เป็นเทคนิคช่วยให้การส่งมอบรวดเร็วและปลอดภัยขึ้น เช่น

- รวมโค้ดบ่อย (CI)
- build/test อัตโนมัติ
- deploy เป็นช่วง ๆ หรืออัตโนมัติ (CD) ทำให้ทีมมั่นใจว่าแต่ละการเปลี่ยนแปลงยังใช้งานได้จริง

8) การสื่อสารและทำงานร่วมกันในทีม (Collaboration)

เทคนิคที่ใช้บ่อย:

-Pair Programming / Peer Review (Code Review)

-การทำงานร่วมกับผู้เกี่ยวข้องอย่างต่อเนื่อง ช่วยลดความผิดพลาดและเพิ่มความเข้าใจร่วมกัน

กระบวนการต่าง ๆ ที่สำคัญ (Key Agile Processes)

1) การวางแผนงาน (Planning)

เป็นขั้นตอนกำหนดเป้าหมายและเลือกงานที่ต้องทำในรอบถัดไป เช่น

- กำหนดเป้าหมาย (Goal) ของรอบงาน
- เลือก User Story ที่สำคัญ
- แตกงานเป็น Task และกำหนดความรับผิดชอบผลลัพธ์คือแผนงานที่ทำได้จริงภายในเวลาที่กำหนด

2) การทำงานประจำวัน (Daily Coordination)

มีการติดตามความคืบหน้าอย่างสั้นและสม่ำเสมอ เช่น Daily meeting (แบบ Scrum) เพื่อ

- บอกว่าวันนี้ทำอะไร
- มีอุปสรรคอะไร
- ต้องการความช่วยเหลือส่วนไหน ทำให้ทีมแก้ปัญหาได้เร็ว ไม่สะสมปัญหา

3) การพัฒนาและการทดสอบ (Development & Testing)

ระหว่างรอบการทำงาน ทีมจะพัฒนาและทดสอบไปพร้อมกัน โดยมักยึดหลัก

- งานต้องผ่านเงื่อนไข Acceptance Criteria
- ตรวจคุณภาพโค้ด/ทดสอบก่อนรวมงาน
- ทำให้ชิ้นงาน “พร้อมใช้งาน” ตามนิยามที่ทีมตกลงกัน (Definition of Done)

4) การทบทวนผลงาน (Review / Demonstration)

เมื่อจบรอบ จะมีการนำเสนอผลงานที่ทำเสร็จให้ผู้เกี่ยวข้องดู เพื่อรับ Feedback จริงจากการใช้งาน/เดโม่ Feedback ที่ได้จะถูกนำกลับไปปรับปรุงและจัดลำดับใหม่ใน Backlog

5) การปรับปรุงกระบวนการทำงาน (Retrospective)

เป็นการประชุมถอดบทเรียนของทีม เพื่อหาว่า

-อะไรที่ทำได้ดีและควรรักษาไว้

-อะไรที่เป็นปัญหา/คอขวด

-รอบหน้าจะปรับอะไรให้ดีขึ้น (Action items)กระบวนการนี้ทำให้ทีมพัฒนา “วิธีทำงาน” ให้มีประสิทธิภาพขึ้นเรื่อย ๆ

6) การส่งมอบ/นำขึ้นใช้งาน (Release/Deployment)

Agile เน้นส่งมอบเป็นระยะ (ไม่รอรวมทีเดียว) โดยอาจทำเป็น

- release เป็นเวอร์ชันย่อย ๆ
- deploy แล้วเก็บข้อมูลการใช้งาน/ปัญหาเพื่อวนกลับไปปรับปรุงในรอบถัดไป